

HULK

Havana University Language for Kompilers

Proyecto de Compilación

Implementación de un Compilador para HULK

Lenguajes de Programación y Compilación

Integrantes

Abel de la Noval Pérez	<i>Grupo C-311</i>
Frank Alberto Piz Torriente	<i>Grupo C-311</i>
Darían Santamarina Hernández	<i>Grupo C-311</i>

Universidad de La Habana
28 de junio de 2026

Capítulo 0

Índice general

1. Introducción	3
1.1. Objetivos del proyecto	3
1.2. El lenguaje HULK	3
1.3. Extensiones implementadas	3
1.4. Estructura del documento	4
2. Arquitectura General del Compilador	5
2.1. Visión general y pipeline	5
2.2. Frontend	5
2.3. Backend	6
3. Análisis Léxico	7
3.1. Implementación	7
3.2. Clasificación de Tokens	7
3.2.1. Tokens específicos de las extensiones	8
3.3. Lookahead y operadores compuestos	8
3.4. Manejo de Errores	8
4. Análisis Sintáctico	9
4.1. Estrategia	9
4.2. Referencias de tipo	9
4.3. Construcciones del AST	9
4.4. Precedencia y asociatividad	10
4.5. Recuperación de errores	10
5. Análisis Semántico	11
5.1. Visión general	11
5.2. Tipo Normalizado	11
5.3. Tabla de Símbolos	12
5.4. Sistema de Tipos y compatibilidad	12
5.4.1. Verificación de protocolos	12
5.4.2. Verificación de iterables	12
5.4.3. Verificación de funtores	12
5.5. Verificador de Expresiones	13
5.6. Manejo de Errores	13
6. Representación Intermedia	14
6.1. Motivación y diseño	14
6.2. Jerarquía del IR	14
6.3. Proceso de Lowering	15
6.3.1. Lowering de expresiones básicas	15

6.3.2. Lowering de estructuras de control	15
6.3.3. Lowering del for loop e iterables	15
6.4. Control Flow Graph	16
6.5. Instrucciones del IR	16
6.6. Vtables y despacho dinámico	17
7. Generación de Código	18
7.1. Arquitectura del backend	18
7.2. Mapeo de tipos y ABI	18
7.3. Orden de emisión	19
7.4. Runtime de Rust	19
7.5. Pipeline de producción del ejecutable	20
8. Extensiones al Lenguaje HULK	21
8.1. Protocolos	21
8.2. Iterables y Generadores	23
9. Comparación General de Diseño	26
9.1. Verificación estructural como hilo conductor	26
9.2. El patrón generador frente a otras alternativas	26
9.3. Dispatch y consistencia del backend	27
10. Conclusiones	28
10.1. Resultados obtenidos	28
10.2. Trabajo futuro	29

Capítulo 1

Introducción

1.1 Objetivos del proyecto

Este trabajo documenta el diseño e implementación de un compilador para el lenguaje **HULK** (*Havana University Language for Kompilers*), desarrollado como proyecto de la asignatura de Lenguajes de Programación y Compilación.

El compilador implementa una cadena completa de compilación que transforma programas HULK en ejecutables nativos mediante LLVM IR como representación intermedia. Las etapas cubiertas comprenden el análisis léxico, sintáctico, semántico, generación de representación intermedia (IR) y generación de código, todo implementado en **Rust**.

Enfoque del informe

Este informe prioriza las decisiones de diseño sobre las explicaciones algorítmicas estándar, con especial atención a las extensiones completadas: **Protocolos e Iterables con generadores**.

1.2 El lenguaje HULK

HULK es un lenguaje orientado a objetos con enfoque docente. Incluye herencia nominal, polimorfismo, inferencia de tipos, y su característica más relevante desde la perspectiva de implementación: **orientación a expresiones**. Casi todas las construcciones sintácticas producen un valor, lo que influye directamente en el análisis semántico y en el proceso de *lowering* hacia IR.

El sistema de tipos combina anotaciones explícitas opcionales con inferencia, sobre una jerarquía nominal como base para las verificaciones semánticas.

1.3 Extensiones implementadas

Las siguientes extensiones amplían las capacidades del lenguaje base:

Extensión	Sintaxis	Mecanismo	Estado
Protocolos	<code>protocol P { m(): T; }</code>	Vtable, dispatch dinámico	Completo
Iterables	<code>T*, for (x in e)</code>	Duck typing, vtable	Completo
Vectores	<code>[e1,e2], [e x in it]</code>	Array dinámico	En desarrollo
Funtores	<code>(T)->U, function (x:T):U ->e</code>	Primera clase	En desarrollo

Cuadro 1.1: Extensiones del lenguaje HULK.

Cada extensión completada requirió modificaciones en el lexer, el parser, el sistema de tipos, la generación de IR y el runtime de Rust.

1.4 Estructura del documento

Los capítulos siguientes describen cada etapa del compilador en orden de ejecución, señalando decisiones de diseño y la incidencia de las extensiones. El capítulo final detalla las extensiones completadas de forma individual.

Capítulo 2

Arquitectura General del Compilador

2.1 Visión general y pipeline

El compilador sigue una arquitectura de fases lineales con separación entre *frontend* (análisis y validación) y *backend* (generación de código), desacopladas por una representación intermedia propia denominada **BANNER IR**.

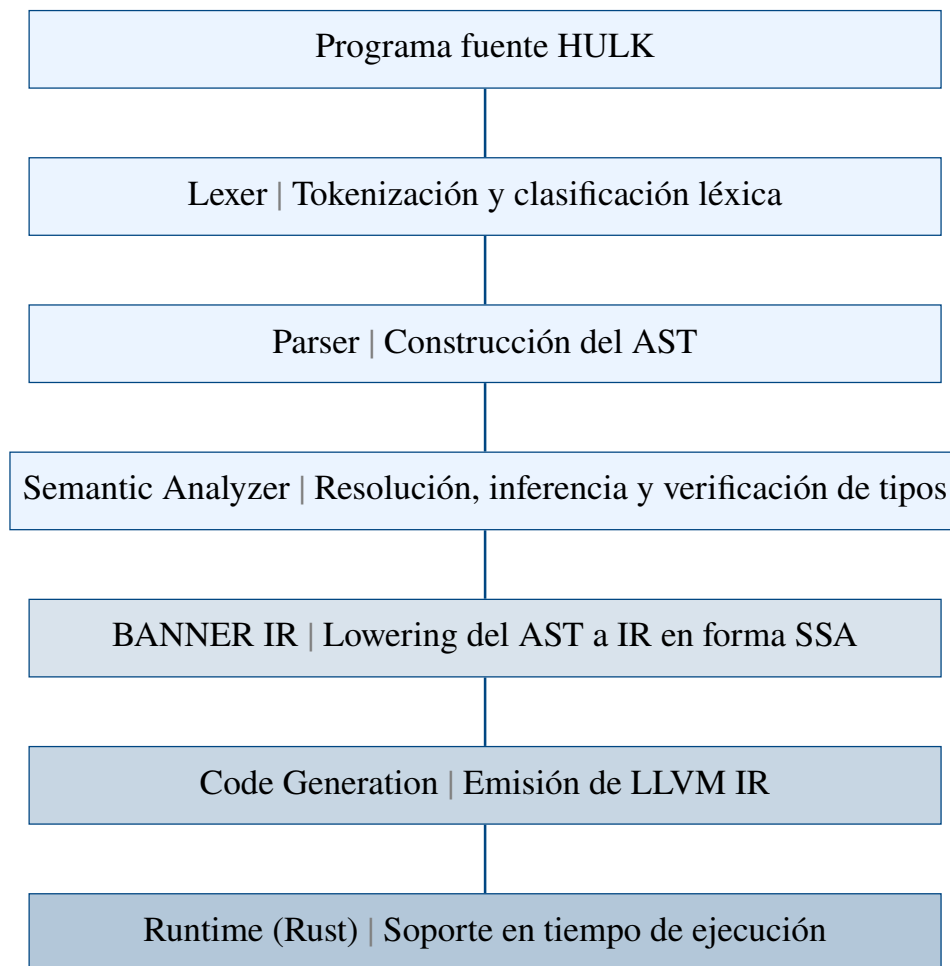


Figura 2.1: Pipeline de compilación.

2.2 Frontend

El *frontend* comprende el lexer, el parser y el analizador semántico. Su salida es un AST completamente anotado con tipos, libre de ambigüedades y validado semánticamente.

Decisión de diseño

Las extensiones de tipos (protocolos, iterables) se integran en el sistema de tipos como variantes del enum `NormalizedType`, evitando tratarlos como casos especiales en etapas posteriores. El mismo mecanismo de `vtable` sirve para ambas extensiones.

2.3 Backend

El backend convierte el BANNER IR a LLVM IR textual mediante `LlvmTextBackend`, y opcionalmente a bitcode mediante `LlvmInkwellBackend` (activado con la feature `llvm-verify`). El ejecutable final se obtiene mediante la cadena `opt` → `llc` → `clang` junto con el runtime estático de Rust.

Capítulo 3

Análisis Léxico

3.1 Implementación

El lexer está implementado manualmente en Rust como un Autómata Finito Determinista (AFD) explícito. Recorre el fuente secuencialmente produciendo tokens acompañados de información de localización (*span*: archivo, línea y columna inicial y final), utilizada en todos los diagnósticos posteriores.

3.2 Clasificación de Tokens

Categoría	Ejemplos	Notas
Palabras reservadas	<code>let if else elif while for</code>	Verificadas antes que los identificadores
Tipos y OOP	<code>type inherits new self</code>	Jerarquía nominal
Extensiones	<code>protocol extends is as</code>	Ver §3.2.1
Identificadores	<code>x myVar Point</code>	Deben iniciar con letra
Literales	<code>42 3.14 "hola" true</code>	Constantes PI, E
Aritmética	<code>+ - * /% ^</code>	<code>^</code> para potenciación
Concatenación	<code>@ @@</code>	Con/sin espacio
Relacionales	<code><= >= == !=</code>	
Lógicos	<code>& !</code>	
Asignación	<code>= :=</code>	<code>:=</code> para reasignación
Flechas	<code>=>-></code>	Cuerpos y tipos función
Delimitadores	<code>() { } [] ; , . :</code>	<code>[]</code> para vectores
Especial	<code> </code>	Separador en comprensiones

Cuadro 3.1: Categorías léxicas reconocidas por el compilador.

3.2.1 Tokens específicos de las extensiones

Las extensiones incorporan tokens nuevos que coexisten con el vocabulario base:

- `protocol`, `extends`: declaración de protocolos y herencia entre ellos.
- `is`, `as`: verificación y conversión de tipos (*type check* y *type cast*).
- `->`: separa parámetros del retorno en anotaciones de función, e.g. `(Number) -> Boolean`.
- `||`: separa la expresión elemento de la variable en comprensiones de vector: `[x*2 || x in range(0,5)]`.
- `[, ]`: literales de vector e indexación.

3.3 Lookahead y operadores compuestos

El lexer utiliza inspección de un carácter hacia adelante para distinguir operadores de múltiples caracteres:

`==   !=   <=   >=   :=   =>   ->   @@   ||`

3.4 Manejo de Errores

Los errores léxicos reportan posición exacta. Se detectan: caracteres inválidos, cadenas sin cerrar, secuencias de escape incorrectas, identificadores que inician con dígito o con guion bajo, y comentarios de bloque no cerrados.

Capítulo 4

Análisis Sintáctico

4.1 Estrategia

El parser fue implementado manualmente usando **Recursive Descent Parsing**. Cada producción relevante de la gramática corresponde a una función especializada que construye el nodo AST correspondiente. Esta estrategia facilitó la incorporación de los nuevos nodos para las extensiones sin afectar la estructura general.

4.2 Referencias de tipo

El nodo `TypeReference` del AST soporta cuatro variantes que cubren tanto el lenguaje base como las extensiones:

Variante	Sintaxis en HULK	Uso
<code>Named(String)</code>	<code>Number, MyType</code>	Tipos nominales
<code>Iterable(T)</code>	<code>Number*</code>	Parámetros de tipo iterable
<code>Vector(T)</code>	<code>Number[]</code>	Parámetros de tipo vector
<code>Function(params, T)</code>	<code>(Number) -> Boolean</code>	Tipos de función

Cuadro 4.1: Variantes de `TypeReference` en el AST.

Decisión de diseño — Tipos de primera clase en el AST

Modelar los tipos iterable, vector y función directamente en `TypeReference` permite validarlos de forma uniforme en el análisis semántico sin casos especiales, y normalizar todas las variantes bajo el enum `NormalizedType`.

4.3 Construcciones del AST

El AST distingue entre *declaraciones globales* y *expresiones*. Las declaraciones globales son:

- `FunctionDeclaration`: función nombrada con parámetros y cuerpo.
- `TypeDeclaration`: tipo con atributos, métodos y herencia opcional.
- `ProtocolDeclaration`: protocolo con firmas de métodos y extensiones opcionales.

Los nodos de expresión más relevantes para las extensiones son:

- For {variable, iterable, body}: iteración sobre cualquier tipo iterable.
- VectorLiteral(Vec<Expr>): literal de vector, e.g. [1, 2, 3].
- VectorComprehension {element_expr, binding, iterable}: comprensión de vector.
- Lambda {parameters, return_type, body}: función anónima.
- TypeCheck / TypeCast {expr, type_ref}: operadores is y as.
- IndexAccess {object, index}: acceso por índice a vectores.

4.4 Precedencia y asociatividad

Nivel	Operadores	Asociatividad
1	Asignación :=	Derecha
2	Lógico OR	Izquierda
3	Lógico AND &	Izquierda
4	Igualdad == !=	Izquierda
5	Relacional <= >=	Izquierda
6	Tipo is as	Izquierda
7	Concatenación @ @@	Izquierda
8	Suma + -	Izquierda
9	Producto * /%	Izquierda
10	Potencia ^	Derecha
11	Unario ! -	Derecha
12	Llamada, acceso . []	Izquierda

Cuadro 4.2: Niveles de precedencia implementados en el parser.

4.5 Recuperación de errores

Ante un error sintáctico, el parser registra el diagnóstico y avanza hasta alcanzar un punto de sincronización (;, }, o inicio de nueva declaración), permitiendo reportar múltiples errores en una sola ejecución.

Capítulo 5

Análisis Semántico

5.1 Visión general

El análisis semántico valida que el programa sea significativo dentro de su contexto. Su arquitectura está dividida en cuatro componentes especializados que colaboran mediante un contexto compartido:

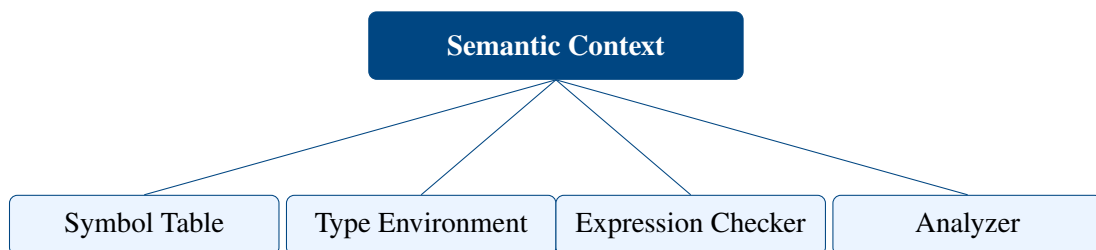


Figura 5.1: Componentes del análisis semántico.

5.2 Tipo Normalizado

Un elemento central del diseño semántico es el enum `NormalizedType`, que representa internamente todos los tipos del sistema con independencia de la sintaxis del fuente:

```
enum NormalizedType {
    Unknown, Number, Boolean, String, Void,
    Named(String), // tipos nominales
    Protocol(String), // tipos protocolo
    Iterable(Box<NormalizedType>), // Number*, Printable*
    Vector(Box<NormalizedType>), // Number[]
    Function(Vec<NormalizedType>, Box<NormalizedType>), // (T)->U
}
```

Decisión de diseño — Representación interna uniforme

Separar la sintaxis del fuente (`TypeReference`) de la semántica interna (`NormalizedType`) permite que los algoritmos de compatibilidad, herencia e inferencia operen sobre una representación canónica sin depender de cadenas de texto ni variaciones sintácticas.

5.3 Tabla de Símbolos

La tabla de símbolos es jerárquica (*stack of scopes*). Al entrar a un nuevo bloque, función, método o tipo se abre un ámbito; al salir, se cierra. Las búsquedas proceden del ámbito más interno al global, implementando *shadowing*.

Categoría	Información almacenada
Variable	Nombre, NormalizedType, span
Parameter	Nombre, tipo, ámbito de función
Function	Firma (parámetros + tipo), tipo de retorno
Type	Definición, herencia, atributos, métodos
Protocol	Firmas de métodos requeridos

Cuadro 5.1: Entidades registradas en la tabla de símbolos.

5.4 Sistema de Tipos y compatibilidad

El *Type Environment* administra las relaciones entre tipos, incluyendo herencia, conformidad con protocolos y compatibilidad estructural de iterables. Sus operaciones principales son el registro de tipos en tiempo de declaración, la resolución de *TypeReference* a *NormalizedType*, el cálculo de conformidad (subtipo, protocolo, iterable) y el cómputo del LCA de la jerarquía para ramas de *if*.

5.4.1 Verificación de protocolos

Un tipo *T* conforma al protocolo *P* si implementa todos los métodos declarados en *P* con firmas compatibles. Esta verificación es **estructural** (no nominal): no se requiere que *T* declare explícitamente que implementa *P*.

Decisión de diseño — Protocolos estructurales

Adoptar verificación estructural (al estilo Go o TypeScript) en lugar de declaración explícita simplifica el uso del lenguaje y permite que tipos existentes satisfagan protocolos nuevos sin modificación. La *vtable* generada es la misma independientemente de si la conformidad fue verificada nominalmente o estructuralmente.

5.4.2 Verificación de iterables

Un tipo es iterable si implementa los métodos `next(): Boolean` y `current(): T` para algún tipo *T*. La anotación *T** en parámetros acepta cualquier tipo que satisfaga esta condición.

El builtin `range(lo, hi)` devuelve *Number**, haciendo que el `for` loop sea composable tanto con la función estándar `range` como con cualquier generador definido por el usuario.

5.4.3 Verificación de funtores

El tipo $(T1, T2) \rightarrow U$ es un tipo de primera clase. Las expresiones lambda son inferidas con este tipo. La compatibilidad de funtores es covariante en el retorno y contravariante en los

parámetros.

5.5 Verificador de Expresiones

El *Expression Checker* recorre el AST de forma descendente con inferencia ascendente: se infiere el tipo de subexpresiones antes de validar la operación del nodo padre. Las verificaciones más relevantes para las extensiones son:

- **For**: la expresión iterable debe ser `Iterable(T)`; la variable toma tipo `T`.
- **VectorLiteral**: todos los elementos deben tener el mismo tipo base; el resultado es `Vector(T)`.
- **VectorComprehension**: el iterable debe ser `Iterable(T)`; produce `Vector(U)`.
- **Lambda**: el retorno se infiere del cuerpo; produce `Function(params, ret)`.
- **IndexAccess**: requiere `Vector(T)`; produce `T`.
- **TypeCheck/TypeCast**: se valida contra el entorno de tipos.

5.6 Manejo de Errores

Los errores semánticos se acumulan en el contexto en lugar de abortar al primer fallo. Se generan diagnósticos para: variables y tipos no declarados, redefiniciones, incompatibilidad de tipos, aridad incorrecta, condiciones no booleanas, y violaciones de conformidad con protocolos e iterables.

Capítulo 6

Representación Intermedia

6.1 Motivación y diseño

Una vez validado el AST se genera una representación independiente de la sintaxis: la **BANNER IR**, inspirada en LLVM IR. Sus propiedades clave son: forma SSA, organización en bloques básicos, CFG explícito, instrucciones de tres direcciones y soporte para llamadas indirectas vía vtable.

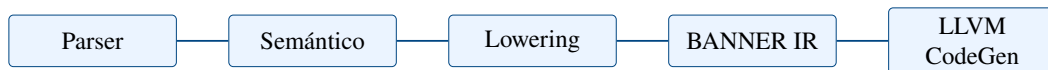


Figura 6.1: Posición de BANNER IR en el pipeline.

6.2 Jerarquía del IR

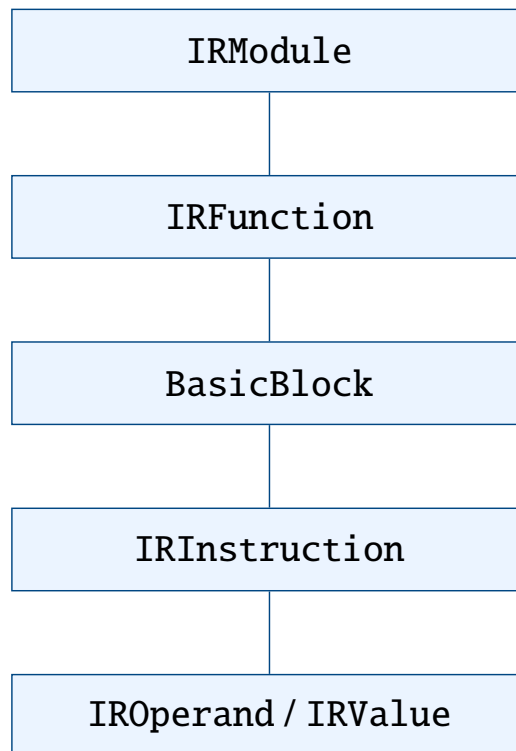


Figura 6.2: Jerarquía estructural de BANNER IR.

El `IRModule` agrupa funciones y globales. Las `IRFunction` contienen firma y bloques básicos. Los `BasicBlock` mantienen listas de instrucciones junto con predecesores y sucesores. Las instrucciones operan sobre `IROperand` (constante, global, o referencia SSA) e `IRValue` (valor SSA con tipo y metadatos).

Los identificadores SSA siguen la convención `%t0, %t1, ...` para temporales, `%p0, %p1, ...` para parámetros, y `bb0, bb1, ...` para bloques.

6.3 Proceso de Lowering

El lowering transforma el AST tipado en BANNER IR mediante un recorrido recursivo descendente que produce instrucciones en el bloque activo y devuelve un `IRValueId` representando el resultado de cada expresión.

El contexto de lowering mantiene: el módulo en construcción, la función y bloque activos, el generador de nombres SSA (`SSAValueGenerator`), el entorno variables→valores SSA, y las tablas de `vtables` por tipo y protocolo.

Decisión de diseño — CFG incremental

El CFG se construye simultáneamente al lowering: al aparecer un `if`, un `while` o un `for`, se crean los bloques necesarios y las aristas de control de inmediato, sin análisis posterior sobre el código emitido.

6.3.1 Lowering de expresiones básicas

Las literales se convierten directamente en `IROperand` constantes. Para `a + b * c`, el AST ya refleja la precedencia; el lowering simplemente sigue su estructura:

```
%t0 = mul b, c
%t1 = add a, %t0
```

6.3.2 Lowering de estructuras de control

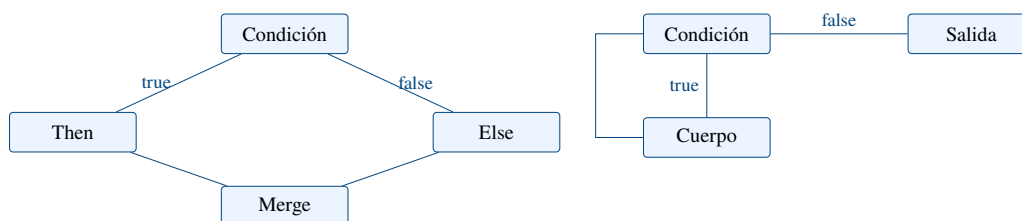


Figura 6.3: CFG generado para `if-else` (izquierda) y `while` (derecha).

6.3.3 Lowering del `for` loop e iterables

El `for` loop es la construcción más relevante del IR para la extensión de Iterables. Su traducción genera tres bloques básicos y dispatch dinámico por `vtable`:

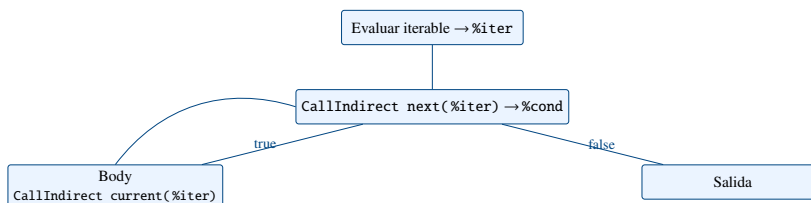


Figura 6.4: Lowering del for loop mediante despacho por vtable.

Decisión de diseño — Dispatch uniforme para iterables

En lugar de generar código especializado por tipo iterable, el for loop siempre emite `CallIndirect` sobre la vtable del objeto. Cualquier tipo con `next()` y `current()` funciona automáticamente, incluidos generadores definidos por el usuario, sin ningún cambio en el lowering.

6.4 Control Flow Graph

La estructura `ControlFlowGraph` encapsula los bloques básicos con sus relaciones de precedencia. Cada `BasicBlock` mantiene listas de predecesores y sucesores. La estructura provee recorridos DFS, BFS y PostOrder (necesario para construcción de dominadores y renombrado SSA), además de validación interna de consistencia y detección de bloques inalcanzables.

6.5 Instrucciones del IR

Instrucción	Descripción
<code>Binary(op, lhs, rhs)</code>	Aritmética, lógica, relacional, concatenación
<code>Unary(op, operand)</code>	Negación aritmética o lógica
<code>Assign(target, src)</code>	Asignación SSA
<code>Call(callee, args)</code>	Llamada directa a función
<code>CallIndirect(fptr, args)</code>	Llamada indirecta (vtable, funtores)
<code>Branch(cond, true_bb, false_bb)</code>	Salto condicional
<code>Jump(bb)</code>	Salto incondicional
<code>Return(value)</code>	Retorno de función
<code>Load / Store</code>	Acceso a memoria
<code>GetElementPtr(base, indices)</code>	Aritmética de punteros (vtable, vectores)
<code>Phi(pairs)</code>	Convergencia de valores SSA

Cuadro 6.1: Conjunto de instrucciones de BANNER IR.

6.6 Vtables y despacho dinámico

Durante el lowering se construyen vtables para todos los tipos con métodos y para todos los protocolos como globales de punteros a función:

```
; Vtable para un tipo con metodos area() y name()
@__vtable_Square = global [2 x ptr] [
    ptr @Square_area,
    ptr @Square_name
]
```

El despacho de un método sobre una variable de tipo protocolo o iterable sigue cuatro pasos: **Load** del puntero de vtable (offset 0 del objeto), **GetElementPtr** al slot del método, **Load** del puntero a función, y finalmente **CallIndirect**. Este mecanismo es idéntico para protocolos e iterables, unificando el tratamiento de ambas extensiones en el backend.

Capítulo 7

Generación de Código

7.1 Arquitectura del backend

El backend está organizado alrededor de un trait `CodegenBackend` que abstrae distintas estrategias de emisión:

```
trait CodegenBackend {  
  fn backend_name(&self) -> &'static str;  
  fn emit_module(module: &IRModule,  
                 context: &CodegenContext) -> CodegenResult<  
    CodegenArtifact>;  
}
```

Se implementan dos backends:

- **LlvmTextBackend**: emite LLVM IR textual (.ll) mediante construcción de cadenas. No requiere dependencias externas y es el backend por defecto.
- **LlvmInkwellBackend**: usa los bindings `inkwell` para la API de LLVM (activado con la feature `llvm-verify`). Permite verificación formal del módulo IR antes de emitirlo.

El `CodegenContext` encapsula la configuración del backend: nombre del módulo, target triple, data layout, y un flag para emisión de comentarios diagnósticos en el IR generado.

Decisión de diseño — Dos backends complementarios

Separar la emisión textual de la bindings-based permite que el compilador funcione sin instalar LLVM durante el desarrollo y las pruebas, mientras que el backend `inkwell` habilita la validación formal del IR en entornos donde LLVM está disponible.

7.2 Mapeo de tipos y ABI

El backend mapea los tipos de BANNER IR a tipos LLVM siguiendo convenciones fijas:

Tipo HULK	Tipo LLVM	Representación
Number	double	IEEE 754 de 64 bits
Boolean	i1	Entero de 1 bit
String	ptr	Puntero a buffer en heap
Objetos / referencias	ptr	Puntero opaco al heap
Tamaños e índices	i64	Entero de 64 bits con signo

Cuadro 7.1: Mapeo de tipos HULK a tipos LLVM.

Los parámetros escalares se pasan directamente por valor. Los objetos y cadenas se pasan como punteros (`ptr`). El target por defecto es `x86_64-unknown-linux-gnu` con el data layout estándar de 64 bits.

7.3 Orden de emisión

El backend emite el módulo LLVM IR en el siguiente orden:

1. **Cabecera:** `ModuleID`, `target triple`, `datalayout`.
2. **Vtable globales:** arreglos `@__vtable_TypeName` para cada tipo y protocolo.
3. **Declaraciones runtime:** funciones externas provistas por el runtime de Rust.
4. **Prototipos externos:** declaraciones de funciones forward-referenced.
5. **Definiciones:** funciones con sus bloques básicos e instrucciones.

El backend detecta y evita emitir declaraciones duplicadas: si una función aparece primero como declare y luego se define, la declaración previa se marca como definida y no se re-emite.

7.4 Runtime de Rust

El runtime es una biblioteca estática (`libruntime_rs.a`) escrita en Rust y compilada con `cargo build -release`. Expone funciones C-compatible (`#[no_mangle] pub extern C fn`) con las siguientes categorías:

Categoría	Funciones
Memoria	<code>hulk_alloc(i64)→ptr</code> , <code>hulk_free(ptr)</code>
Cadenas	<code>hulk_concat</code> , <code>hulk_concat_space</code> , <code>hulk_num_to_str</code> , <code>hulk_strlen</code>
Impresión	<code>print_number</code> , <code>print_string</code> , <code>print_bool</code> , <code>print_object</code>
Iterables	<code>range(double, double, ptr)→ptr</code> , <code>Range_next(ptr)→i1</code> , <code>Range_current(ptr)→double</code>
Vectores	<code>hulk_vector_new</code> , <code>hulk_vector_push</code> , <code>hulk_vector_get</code> , <code>hulk_vector_size</code>
Matemáticas	<code>pow(double, double)→double</code> , <code>strcmp(ptr, ptr)→i32</code>

Cuadro 7.2: Funciones del runtime de Rust expuestas al código LLVM IR generado.

El tipo `Range` en el runtime es una estructura C con un puntero de `vtable` en el offset 0 (para compatibilidad con el dispatch dinámico), seguido de los campos `min`, `max` y `current`. La función `range(lo, hi, vtable)` recibe la `vtable` del tipo `Range` como parámetro para construir el objeto iterable en el heap.

Decisión de diseño — Runtime en Rust con ABI C

Implementar el runtime en Rust y exponerlo como biblioteca C estática garantiza seguridad de memoria en las operaciones críticas (gestión de heap, strings, vectores dinámicos) sin imponer un GC ni un overhead de FFI. El campo `vtable` en el offset 0 de `Range` permite que el mismo mecanismo de dispatch dinámico usado para tipos definidos por el usuario funcione también con el builtin `range()`.

7.5 Pipeline de producción del ejecutable

Una vez emitido el LLVM IR textual, el proceso de construcción del ejecutable sigue:



Figura 7.1: Pipeline de construcción del ejecutable nativo.

`opt` aplica optimizaciones estándar (`-O2`). `llc` genera código objeto con modelo de reubicación PIC. `clang` enlaza el objeto con el runtime estático de Rust (`libruntime_rs.a`) para producir el ejecutable final.

Capítulo 8

Extensiones al Lenguaje HULK

8.1 Protocolos

Motivación

En el lenguaje base, la única forma de lograr polimorfismo es mediante herencia de tipos. Esto impone un acoplamiento estructural fuerte: un tipo debe ser subclase de otro para poder ser tratado como él, lo que no siempre refleja relaciones conceptuales naturales.

Los **protocolos** resuelven este problema permitiendo definir contratos de comportamiento que cualquier tipo puede satisfacer sin necesidad de herencia explícita. De esta forma, tipos completamente independientes en la jerarquía pueden compartir una interfaz común y ser tratados de manera polimórfica.

Diseño sintáctico

Un protocolo se declara con la palabra clave `protocol` y contiene firmas de métodos sin implementación:

```
protocol Shape {
  area(): Number;
  name(): String;
}

// Los protocolos pueden extender otros protocolos
protocol Drawable extends Shape {
  draw(): String;
}
```

Un tipo satisface un protocolo simplemente implementando los métodos requeridos, sin ninguna declaración de conformidad:

```
type Square(s: Number) {
  side: Number = s;
  area(): Number { self.side * self.side; }
  name(): String { "square"; }
}

// Square satisface Shape automaticamente
let sq: Shape = new Square(4) in sq.area(); // => 16
```

Las variables de tipo protocolo aceptan cualquier objeto que conforme el protocolo, y los métodos se despachan dinámicamente mediante `vtable`.

Implementación

En el parser, `ProtocolDeclaration` captura tres elementos: el nombre del protocolo, la lista de protocolos extendidos (campo `extends`, de tipo `Vec<TypeReference>`) y las firmas de métodos (campo `members`, de tipo `Vec<ProtocolMethodSignature>`). Una firma incluye nombre, parámetros tipados y tipo de retorno.

En el analizador semántico, los protocolos se registran en el *Type Environment* al inicio del análisis (en un primer paso de recolección de declaraciones), lo que permite referencias forward entre protocolos y tipos. La verificación de conformidad es estructural: para cada método m requerido por P , se comprueba que el tipo T posea un método con el mismo nombre y firma compatible.

Si un protocolo P extiende a Q , todos los métodos de Q se requieren también en P , y cualquier tipo que conforme P debe satisfacer también Q .

En el lowering, cada protocolo genera una tabla de índices de métodos. Cuando se accede a un método sobre una variable de tipo protocolo, se consulta `get_protocol_method_index(proto, method)` para obtener el índice de `vtable` y emitir la secuencia `Load / GetElementPtr / Load / CallIndirect`.

Análisis semántico — detalles

La conformidad de un tipo T con un protocolo P se verifica en dos momentos:

1. Al **asignar** un objeto a una variable de tipo protocolo: `let x: P = new T(...)`.
2. Al **pasar** un argumento de tipo T a una función que espera un parámetro de tipo P .

En ambos casos, el *Type Environment* itera sobre los métodos requeridos por P y verifica que T (o algún ancestro en la jerarquía de herencia) los implemente. Si la comprobación falla, se emite un error semántico indicando qué métodos faltan.

Decisión de diseño — Conformidad por herencia incluida

Si un tipo hereda de otro que ya satisface el protocolo, la herencia transitiva es suficiente: no se necesita re-declarar la conformidad en cada subclase. Esto es coherente con el enfoque estructural y reduce la verbosidad en jerarquías profundas.

Comparación con otros lenguajes

Aspecto	HULK	Java	Go / TypeScript
Declaración	<code>protocol</code>	<code>interface</code>	<code>interface</code>
Conformidad	Estructural (impl.)	Nominal (<code>implements</code>)	Estructural (implícita)
Herencia entre protoc.	<code>extends</code>	<code>extends</code>	<code>interface A extends B</code>
Dispatch	<code>Vtable</code>	<code>Vtable (JVM)</code>	<code>Vtable (Go) / prototipado (TS)</code>

Cuadro 8.1: Comparación de protocolos con interfaces en otros lenguajes.

La principal diferencia con Java es que en HULK no se necesita declarar `implements P`: basta con implementar los métodos. Esto se asemeja más a Go y TypeScript, con la diferencia

de que HULK usa despacho por vtable (similar a Go) en lugar de prototipado dinámico (TypeScript/JavaScript).

Discusión

El enfoque estructural tiene ventajas claras en términos de expresividad: tipos de bibliotecas externas o tipos ya definidos pueden satisfacer protocolos nuevos sin modificación. Sin embargo, introduce una consideración: en lenguajes con conformidad nominal es explícito qué protocolos implementa un tipo; en HULK esto debe inferirse examinando los métodos del tipo.

Desde la perspectiva de implementación, el dispatch dinámico por vtable es eficiente y predecible, con un costo por llamada de tres instrucciones de carga de memoria más la llamada indirecta, igual al costo de los métodos virtuales en C++.

8.2 Iterables y Generadores

Motivación

El patrón de iteración es uno de los más frecuentes en programación. En HULK base, iterar sobre una colección requería construir manualmente índices o usar recursión. La extensión de Iterables introduce un mecanismo de iteración uniforme que funciona sobre cualquier tipo que provea una interfaz de avance y consulta del elemento actual, sin necesidad de herencia.

El diseño tomó como referencia el patrón *generador*: en lugar de materializar todos los elementos en memoria, el objeto iterable produce elementos bajo demanda en cada llamada a `next()`.

Diseño

La interfaz de iterable es implícita: cualquier tipo con los métodos `next(): Boolean` y `current(): T` es un iterable de elementos de tipo T.

```
// Generador definido por el usuario
type OddNumbers(limit: Number) {
  current_val: Number = -1;
  limit_val: Number = limit;

  next(): Boolean {
    self.current_val := self.current_val + 2;
    self.current_val <= self.limit_val;
  }
  current(): Number { self.current_val; }
}

// El for loop acepta cualquier iterable
for (x in new OddNumbers(9)) {
  print(x); // imprime 1 3 5 7 9
}
```

La anotación de tipo `T*` expresa “iterable de T” y puede usarse en parámetros de función:


```
// Funcion generica sobre cualquier iterable de Number
function sum_gen(gen: Number*): Number {
  let s: Number = 0 in {
    for (x in gen) { s := s + x; };
    s;
  };
}

sum_gen(new OddNumbers(9)); // => 25
sum_gen(range(1, 6)); // => 15
```

El builtin `range(lo, hi)` devuelve un `Number*` y está implementado directamente en el runtime de Rust.

Implementación

En el lexer y parser, el sufijo `*` en una referencia de tipo produce la variante `Iterable(T)` del enum `TypeReferenceKind`. El nodo `ExprKind::For` captura la variable de iteración, la expresión iterable y el cuerpo.

En el análisis semántico, la verificación del `for` comprueba que la expresión iterable tenga tipo `Iterable(T)` (o que sea un tipo con los métodos `next()` y `current()` correctos, en cuyo caso se infiere el tipo elemento). La variable de iteración se liga al tipo `T` dentro del ámbito del cuerpo.

La función `range` se registra como builtin con firma `(Number, Number) → Number*`, lo que permite su uso directo en expresiones `for` sin declaración explícita.

En el lowering, el `for` se traduce siempre mediante `CallIndirect` sobre la vtable del objeto iterable (ver Figura 6.4). Para el builtin `range`, el lowering emite una llamada a `range(lo, hi, @__vtable_Range)`, pasando el puntero de la vtable de `Range` para que el dispatch dinámico funcione correctamente. Para tipos usuario, se usa la vtable del tipo concreto construida durante el lowering.

Decisión de diseño — Generadores sin interfaz declarada

No se exige que un tipo declare explícitamente que es iterable (no hay equivalente a `implements Iterable<T>`). La verificación es puramente estructural: el analizador comprueba la presencia y firma de `next()` y `current()`. Esto permite definir generadores como tipos ordinarios de HULK sin ninguna sintaxis adicional.

Comparación con otros lenguajes

Aspecto	HULK	Python	Java
Interfaz	<code>next()</code> <code>current()</code> (estructural)	+ <code>__iter__</code> + <code>__next__</code>	+ <code>Iterable<T></code> (nominal)
Declaración	Ninguna	Ninguna (duck typing)	<code>implements Iterable<T></code>
Valor de avance	<code>next(): Boolean</code>	<code>StopIteration</code>	<code>hasNext(): boolean</code>
Valor actual	<code>current(): T</code>	<code>__next__(): T</code>	<code>next(): T</code>
For loop	<code>for (x in e)</code>	<code>for x in e</code>	<code>for (T x : e)</code>
Builtin rango	<code>range(lo, hi)</code>	<code>range(lo, hi)</code>	<code>IntStream.range(lo, hi)</code>

Cuadro 8.2: Comparación del mecanismo de iteración en diferentes lenguajes.

HULK combina la simplicidad del duck typing de Python (sin declaración de conformidad) con la tipificación estática de Java (el tipo del elemento se conoce en tiempo de compilación). La separación de `next()` (avanza y retorna bool) y `current()` (retorna el valor actual) difiere de Python donde `__next__` combina ambas acciones.

Discusión

La principal ventaja del diseño es su uniformidad: el `for` loop, el builtin `range` y los generadores de usuario usan exactamente el mismo mecanismo de dispatch. El compilador no necesita conocer qué tipo concreto se itera; el dispatch dinámico por `vtable` lo resuelve en tiempo de ejecución con un costo constante.

El costo de este diseño es que cada iteración implica dos llamadas indirectas (`next` y `current`), a diferencia de la iteración estática sobre arreglos, que sería un acceso directo indexado. Para la mayoría de los casos de uso docente esto es aceptable; en un compilador de producción se podría aplicar *devirtualización* cuando el tipo concreto es conocido en el sitio de llamada.

Capítulo 9

Comparación General de Diseño

9.1 Verificación estructural como hilo conductor

Tanto los protocolos como los iterables de HULK descansan sobre el mismo principio: la verificación estructural. En lugar de exigir que un tipo declare explícitamente qué contratos satisface, el compilador comprueba si posee los métodos requeridos con las firmas correctas. Esta elección sitúa a HULK más cerca de Go y TypeScript que de Java.

Go fue uno de los primeros lenguajes de uso general ampliamente adoptados en llevar esta idea hasta sus consecuencias naturales: cualquier tipo que posea los métodos de una interfaz la satisface, sin una sola palabra clave adicional. TypeScript adopta el mismo principio pero en un sistema estructural completo, donde incluso la forma de objetos literales anónimos es suficiente para cumplir una interfaz. HULK se sitúa entre ambos: como Go, la verificación es estática y ocurre en tiempo de compilación; como TypeScript, las anotaciones de tipo son explícitas aunque opcionales.

La diferencia con Java es la más instructiva. Java exige `implements NombreInterfaz` en la declaración del tipo, lo que tiene ventajas claras: la intención es visible en el código y los entornos de desarrollo pueden razonar sobre la conformidad sin analizar el cuerpo del tipo. El inconveniente es la rigidez: un tipo de biblioteca externa no puede satisfacer una interfaz nueva sin modificar su código fuente. El enfoque de HULK es más flexible, pero requiere inspeccionar los métodos de un tipo para saber qué protocolos satisface, algo que en Java es inmediato leyendo su declaración.

9.2 El patrón generador frente a otras alternativas

El mecanismo de iteración de HULK — dos métodos separados `next()` y `current()` — es deliberadamente más explícito que el de Python, donde `__next__` avanza y devuelve el valor actual en una sola llamada, señalando el fin mediante una excepción `StopIteration`. La separación entre avance y consulta tiene una ventaja directa en el backend: el `for` loop emite exactamente dos `CallIndirect` por iteración con semántica clara, sin necesidad de manejar excepciones ni valores centinela.

Comparado con Java, el diseño es más ligero. Java requiere declarar `implements Iterable<T>` e implementar `iterator()`, que a su vez devuelve un `Iterator<T>` con `hasNext()` y `next()`. HULK colapsa esa indirección exigiendo los métodos directamente en el tipo, sin una clase envoltorio separada. El precio es que el protocolo de iterable de HULK es una convención implícita verificada por el compilador, no un tipo nominal que aparezca en la firma del tipo.

JavaScript y TypeScript tienen su propio protocolo de iterador (`next()` que devuelve `{value,`

`done}`), más cercano al de Python en que combina el valor y la señal de fin en un único retorno. HULK mantiene ambos separados, lo que simplifica el lowering pero obliga al tipo a conservar estado interno entre llamadas consecutivas a `next()` y `current()`.

9.3 Dispatch y consistencia del backend

Una consecuencia directa del uso de vtables para ambas extensiones es que el rendimiento es predecible: cada llamada a un método de protocolo o a `next()`/`current()` tiene el mismo coste — una carga de memoria para obtener la vtable, otra para el puntero a función, y la llamada indirecta. No hay caché de métodos ni búsqueda en cadena de prototipos.

Esto contrasta con TypeScript y JavaScript, donde el motor puede aplicar optimizaciones especulativas agresivas (inline caches, hidden classes), pero a costa de un comportamiento más difícil de predecir en casos límite. Go usa un mecanismo de vtable análogo al de HULK para sus interfaces, lo que explica que ambos tengan perfiles de coste similares para el despacho polimórfico: constante y sin sorpresas.

La uniformidad del mecanismo fue la decisión de diseño más relevante del backend. Significa que agregar nuevas extensiones que requieran polimorfismo no exige modificar el generador de código: basta con registrar los métodos correspondientes en la vtable del tipo durante el lowering.

Característica	HULK	Go	TypeScript
Protocolos / interfaces	Estructural, protocol	Estructural, interface	Estructural, interface
Conformidad	Implícita	Implícita	Implícita
Dispatch polimórfico	Vtable propia	Vtable (iface)	Prototipado JS
Iteración	T*, duck typing	range builtin	Iterable<T>
Generadores	Tipos con next/current	Goroutines/channel	function*

Cuadro 9.1: Resumen comparativo de las extensiones implementadas.

Capítulo 10

Conclusiones

10.1 Resultados obtenidos

El proyecto logró implementar una cadena de compilación completa y funcional para el lenguaje HULK, desde el análisis léxico hasta la generación de ejecutables nativos mediante LLVM. El compilador procesa correctamente el conjunto de construcciones definidas por la especificación base del lenguaje e incorpora dos extensiones completamente integradas en todas las etapas de compilación.

Los resultados más relevantes pueden resumirse en los siguientes puntos:

- **Pipeline completo:** el compilador transforma programas HULK en ejecutables nativos pasando por lexer, parser, análisis semántico, BANNER IR y generación de LLVM IR, con enlace contra el runtime de Rust.
- **BANNER IR:** el diseño de una representación intermedia propia en forma SSA, con CFG explícito y soporte para llamadas indirectas, permitió desacoplar completamente el frontend del backend y unificar el tratamiento de todas las extensiones de tipos bajo el mismo mecanismo de dispatch por vtable.
- **Protocolos:** la implementación de protocolos con verificación estructural demostró ser una adición expresiva al lenguaje. El hecho de que cualquier tipo que implemente los métodos requeridos conforme automáticamente a un protocolo, sin declaración explícita, resultó coherente con la filosofía del lenguaje y no introdujo complejidad adicional en el backend.
- **Iterables y generadores:** la extensión de iterables mediante duck typing estructural logró que el `for` loop funcione de forma uniforme tanto con el builtin `range` como con generadores definidos por el usuario, sin ningún caso especial en el lowering. El tipo anotado `T*` integra la extensión limpiamente en el sistema de tipos.
- **Dos backends de generación:** la separación entre `LlvmTextBackend` e `LlvmInkwellBackend` permitió desarrollar y probar el compilador sin depender de una instalación de LLVM, manteniendo la posibilidad de verificación formal del IR cuando está disponible.
- **Runtime en Rust:** implementar el soporte en tiempo de ejecución en Rust con ABI C garantizó seguridad de memoria en las operaciones críticas (gestión de heap, cadenas, vectores dinámicos) sin introducir un recolector de basura ni overhead de FFI.

Desde el punto de vista de las decisiones de diseño, la elección de verificación estructural para protocolos e iterables resultó acertada: ambas extensiones comparten el mismo mecanismo de vtable en el backend, lo que mantiene el compilador simple y coherente. El costo de esta uniformidad fue la necesidad de implementar lógica de inferencia estructural en el analizador semántico, pero esto quedó localizado en el *Type Environment* sin afectar al resto de la arquitectura.

10.2 Trabajo futuro

Las siguientes líneas de trabajo quedan abiertas para extender y mejorar el compilador:

- **Completar la extensión de Vectores:** la sintaxis de vectores literales y comprensiones está implementada en el parser y el sistema de tipos, pero la integración completa con el runtime y la generación de código requiere trabajo adicional, en particular el manejo de vectores como valores de primera clase en todas las posiciones del lenguaje.
- **Completar la extensión de Funtores:** las lambdas y los tipos función están representados en el AST y el sistema de tipos. El lowering de cierres con captura de entorno y el dispatch de funtores como `CallIndirect` son las piezas que restan para tener funtores completamente funcionales.
- **Optimizaciones sobre BANNER IR:** el IR propio ofrece un punto natural para aplicar pases de optimización antes de la emisión de LLVM IR. Candidatos inmediatos son la propagación de constantes, eliminación de código muerto, y devirtualización cuando el tipo concreto del receptor es conocido en el sitio de llamada (lo que eliminaría el overhead del dispatch por vtable en casos frecuentes como `range`).
- **Mejora del sistema de inferencia de tipos:** la inferencia actual es unidireccional (ascendente). Extenderla con inferencia bidireccional permitiría omitir anotaciones de tipo en más contextos, como en parámetros de lambdas cuando el tipo de función esperado es conocido.
- **Información de depuración:** añadir emisión de metadatos DWARF al LLVM IR generado permitiría depurar programas HULK con herramientas estándar como `gdb` o `lldb`.
- **Biblioteca estándar:** el runtime actual provee operaciones primitivas. Una biblioteca estándar con estructuras de datos (pilas, colas, mapas) implementadas en HULK sobre las extensiones de Iterables y Vectores enriquecería significativamente el lenguaje.